Compiler-Validated Semantic Editing for Weak Local Coding Models

Guy J. Grigsby
guy@grigsby.dev

Draft — July 24, 2026

Abstract

Coding agents edit source code as text, and small locally runnable models fail at it: diff formats break, identifiers get guessed, and errors surface only after files change. ago is a semantic edit protocol for Go. The agent queries a typechecked workspace and submits mutations from a versioned operation catalog; the compiler validates every mutation against an in-memory snapshot before anything reaches disk. Edits apply as atomic, generation-checked transactions or not at all, so an edit that does not typecheck is unrepresentable, not merely checked. Rejections are the conversation: each carries typed diagnostics and paste-ready repairs (the corrected call, verbatim), and resending a rejected call unchanged escalates. An oracle replays each task's ground-truth commit through the protocol before any model attempts it, so a task enters the benchmark only once it is proven solvable; tasks are mined from real commits in traefik, vault, and boundary. The evaluation is a three-arm grid, raw file editing, LSP symbol addressing with advisory validation (Serena), and the protocol, run against a frozen engine so the benchmark cannot chase the tasks. Across three locally served models the arm ordering never inverts: the protocol beats symbol addressing, which beats raw text editing. The floor 9B model passes 41% of episodes through the protocol against 32% with symbol addressing alone and 24% raw; the widest gap is the model that is a poor freeform tool-caller, whose raw pass rate collapses to 4% yet recovers to 53% through the protocol. Sampled on-disk states during editing fail to compile 76%–86% of the time in the raw arm and at most 2% in the semantic arm, in every model. All code, per-episode evidence (transcripts, request logs, serving configurations), and analysis tooling are released.

1 Introduction

TODO(guy): expand from idea.md.

2 Related Work

Symbol-level agent tooling over LSP (Serena [7]; gopls's experimental MCP server [8]) navigates semantically but still edits text, or exposes a single mutation. The compiler diagnostics sit right there in those stacks; the difference here is that validation is not advisory. CODESTRUCT [3] frames agents over structured action spaces with syntax-validated edits and reports token and accuracy gains; ago's mutations are type-checked, transactional, and carry repairs. Repository-scale editing benchmarks (RES-Q [4], FEA-Bench [5], EDIT-Bench [1]) find instructed edits hard for current models, and edit-format leaderboards [2] track a distinct failure where a model

cannot emit a valid edit in the required format at all. That is the failure mode the repair channel converts: the format stays strict and the rejection hands back the corrected call. Compiler-guided patch generation (Repilot [9]) and typed-hole context (Hazel [6]) take the same validate-during-generation stance at expression granularity.

3 The ago Protocol

3.1 Surface

Ten tools carry a versioned catalog of 33 operations across declaration, statement, test, and project families, plus seven query kinds with deterministic, paged responses. Node handles address statements inside declarations; generation counters make staleness explicit instead of silent.

3.2 Transactions

An ordered op list validates as one unit against an in-memory snapshot. Ops apply to a copy, the dirty set re-typechecks once (in parallel, scheduled over post-edit import edges), resolution proofs run (rename rejects reference capture the compiler would happily accept), then all files write together or nothing does. Pre-existing issues in the affected packages are baselined and tolerated; real codebases carry history, and only new diagnostics reject.

3.3 Rejection as conversation

Every rejection answers one question: what is the correct next call. It carries typed diagnostics, candidate lists, and `possible_repairs`, complete calls the agent resends verbatim. Repairs are tested by execution before they are offered; a repair that would itself reject is worse than none. Resending a rejected call unchanged escalates. Accepted mutations embed the fresh view of the touched declaration, so back-to-back edits skip the read round trip.

4 Oracle Methodology

Every benchmark task comes from a real commit, so the correct answer is already known. Before any model attempts a task, the oracle submits that known answer through the protocol itself. If the right answer cannot get through, the model never had a chance: the problem is ours (an engine bug, an extractor error, or a named protocol ceiling), and the task stays out of the roster until it is fixed. This separates "the model failed" from "the task was impossible" by construction, and the oracle transcripts double as a training corpus.

5 Benchmark

Same model, same harness, same prompts (token parity is enforced by test), same time cap, three arms. Raw edits files with shell tools. Serena addresses symbols through the language server (a pinned revision of the Serena MCP server) but validation stays advisory: diagnostics are available on request and nothing blocks a broken write from landing. Semantic sees only the protocol. The middle arm is the ablation that separates the two variables the protocol bundles: symbol addressing and mandatory validation. Scoring is a goal predicate plus a clean

typecheck plus scoped tests, and the test gate is voided where a pristine worktree also fails it. Serving configurations are pinned per run: GLM-4.7-Flash (MoE) at UD-Q4_K_XL, roughly 20 GiB with a q8_0 K/V cache at 128k context; gpt-oss-20b at MXFP4, 11.28 GiB of weights before KV; Qwen3.5-9B at UD-Q4_K_XL, its card's embedding server evicted to fit the larger quant. Local serving is part of the experiment, constraints included. Every episode records its transcript, diff, score, token usage, failure kind, and the daemon's request log; the evidence is the contribution as much as the numbers.

5.1 Evaluation freeze

Every run before the freeze tag is development data: rejections became repairs and oracle failures became engine fixes, a loop that is a contribution but cannot also be the headline evidence. The engine is frozen at a tagged revision (`eval-freeze-1`); every episode records the engine revision it ran under, so the split is auditable. Headline tables come only from frozen-tag runs over the development set of 20 oracle-certified tasks. A held-out set mined from etcd, caddy, and hugo is extracted and certified only after the tag, with the frozen engine; a certification failure there is reported as a protocol ceiling, not fixed.

5.2 Invalid intermediate states

The mechanism claim is that validation before disk changes what a repository passes through, not just where an episode ends. A watcher samples each worktree during the agent's editing window: when the mtime fingerprint of the Go files changes and then holds still for one interval, the settled state gets a `go build ./...` verdict. The oracle runs as the control and measures the sampler itself. Raw and advisory arms pass through whatever the model writes; the semantic arm cannot land a declaration that does not typecheck, so its measured rate is the write window itself: a mutation touching dozens of call sites writes files sequentially, and a sample can catch the half-written state. Replaying every accepted mutation from the flagged episodes shows each settled state compiles; no host tool wrote Go in any of them.

6 Results

Pooled over the frozen grid, semantic passed 173/340 episodes; raw passed 78/340. The per-task table tells the sharper story: raw wins only where grep suffices, and goes to zero on the tasks that need whole-workspace consistency.

Two readings stand out. First, the arm ordering (Figure 1) never inverts across models: symbol addressing helps, and mandatory validation helps again on top of it. Second, the protocol's advantage is not uniform. It is widest for the model that edits text worst: a strong instruction-follower that is nonetheless a poor freeform tool user passes almost nothing raw yet recovers to a majority through the protocol, because the protocol removes exactly the failure it is prone to. The invalid-intermediate rate (Figure 2) is the mechanism: both text arms leave the workspace uncompilable most of the time mid-edit, while the semantic arm holds it valid throughout.

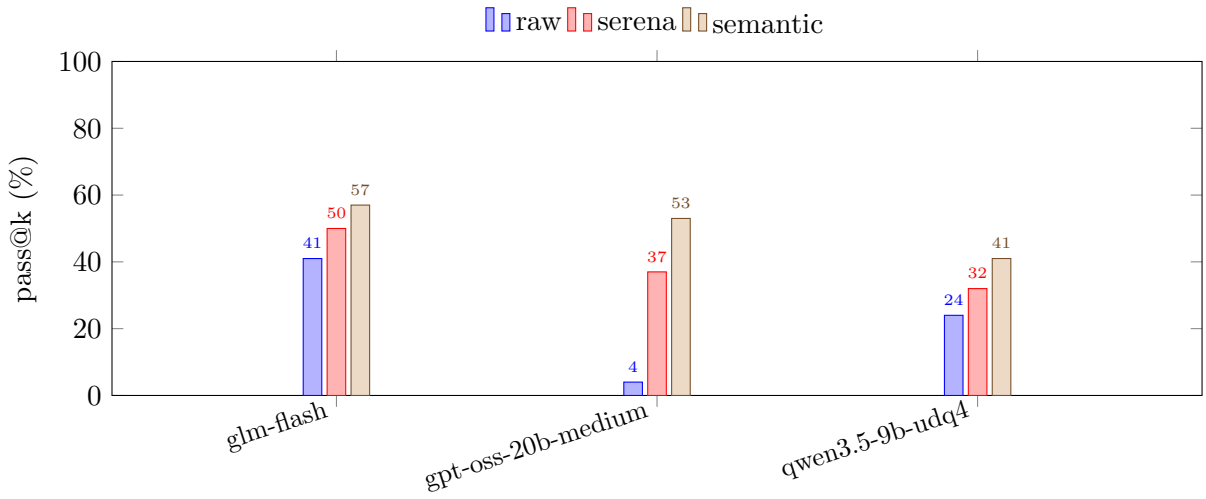

Figure 1: Pass@k by arm, per model. The ordering semantic $\geq$ serena $\geq$ raw holds for every model in the grid. The lift is largest where the model is a weak freeform tool-caller (gpt-oss-20b-medium: 4% raw, 53% semantic). Generated from committed episode evidence.

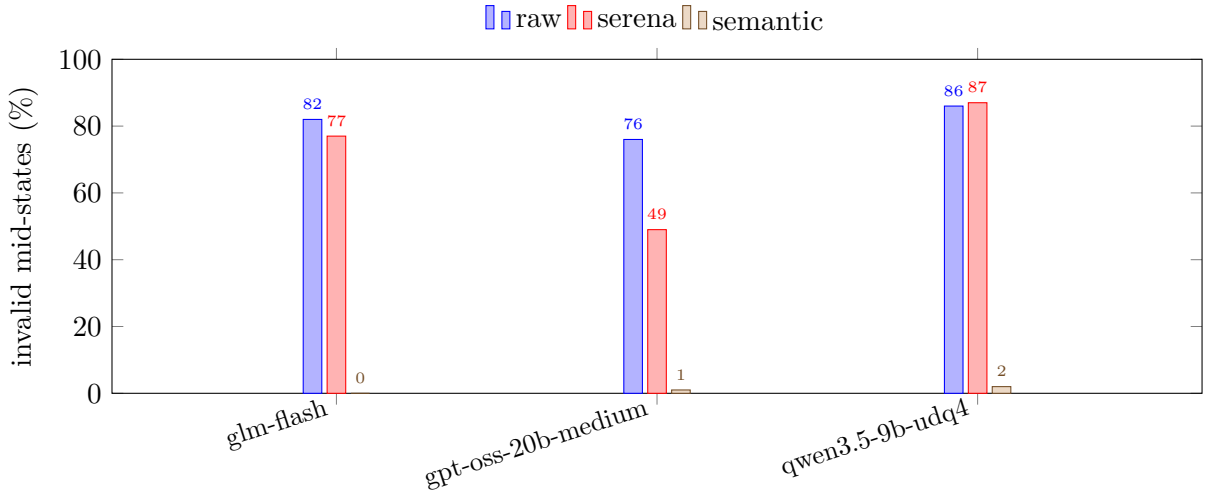

Figure 2: Fraction of sampled on-disk states that fail `go build` during the editing window. The text-editing arms churn through broken states; the semantic arm cannot land a declaration that does not typecheck, so its residual is only the multi-file write window. Generated from the same evidence.

profile	task	mode	pass@k	median green (s)	capped	resends
glm-flash	boundary_06d69f69	raw	6/6	244	0	0
glm-flash	boundary_06d69f69	semantic	6/6	250	0	2
glm-flash	boundary_06d69f69	serena	6/6	178	0	0
glm-flash	boundary_35b91c66	raw	2/6	544	4	0
glm-flash	boundary_35b91c66	semantic	3/6	379	3	12
glm-flash	boundary_35b91c66	serena	5/6	247	0	0
glm-flash	boundary_57ffc718	raw	0/6	–	6	0
glm-flash	boundary_57ffc718	semantic	2/6	469	4	29
glm-flash	boundary_57ffc718	serena	0/6	–	6	0
glm-flash	boundary_687dd1bd	raw	0/6	–	3	0
glm-flash	boundary_687dd1bd	semantic	0/6	–	6	72
glm-flash	boundary_687dd1bd	serena	0/6	–	5	0
glm-flash	boundary_9237d6f7	raw	1/6	554	2	0
glm-flash	boundary_9237d6f7	semantic	3/6	401	2	98
glm-flash	boundary_9237d6f7	serena	5/6	175	0	0
glm-flash	boundary_9354a4eb	raw	0/6	–	6	0
glm-flash	boundary_9354a4eb	semantic	0/6	–	6	11
glm-flash	boundary_9354a4eb	serena	0/6	–	6	0
glm-flash	boundary_9f2c83f3	raw	2/6	671	5	0
glm-flash	boundary_9f2c83f3	semantic	5/6	407	0	6
glm-flash	boundary_9f2c83f3	serena	3/6	675	0	0
glm-flash	boundary_b26814a3	raw	1/6	720	5	0
glm-flash	boundary_b26814a3	semantic	4/6	669	4	30
glm-flash	boundary_b26814a3	serena	6/6	406	0	0
glm-flash	boundary_b4b95e0f	raw	0/6	–	6	0
glm-flash	boundary_b4b95e0f	semantic	0/6	–	6	7
glm-flash	boundary_b4b95e0f	serena	0/6	–	6	0
glm-flash	boundary_bf1486f7	raw	0/6	–	6	0
glm-flash	boundary_bf1486f7	semantic	4/6	657	4	8
glm-flash	boundary_bf1486f7	serena	0/6	–	6	0
glm-flash	boundary_c0532428	raw	6/6	453	1	0
glm-flash	boundary_c0532428	semantic	6/6	334	0	13
glm-flash	boundary_c0532428	serena	6/6	138	0	0
glm-flash	boundary_caf19f86	raw	6/6	715	3	0
glm-flash	boundary_caf19f86	semantic	6/6	325	0	6
glm-flash	boundary_caf19f86	serena	6/6	171	0	0
glm-flash	boundary_dd2c3807	raw	1/6	900	3	0
glm-flash	boundary_dd2c3807	semantic	3/6	720	6	21
glm-flash	boundary_dd2c3807	serena	1/6	741	4	0
glm-flash	boundary_eb61ac63	raw	0/6	–	6	0
glm-flash	boundary_eb61ac63	semantic	4/6	720	6	21
glm-flash	boundary_eb61ac63	serena	0/6	–	6	0
glm-flash	boundary_fb1ea92a	raw	6/6	431	0	0
glm-flash	boundary_fb1ea92a	semantic	0/6	–	6	81
glm-flash	boundary_fb1ea92a	serena	2/6	323	2	0
glm-flash	traefik_51491463	raw	5/6	272	0	0
glm-flash	traefik_51491463	semantic	6/6	268	0	3
glm-flash	traefik_51491463	serena	5/6	128	0	0
glm-flash	vault_7103bc2c	raw	0/6	–	3	0
glm-flash	vault_7103bc2c	semantic	0/6	–	6	20
glm-flash	vault_7103bc2c	serena	0/6	–	5	0
glm-flash	vault_95b24f55	raw	6/6	340	0	0
glm-flash	vault_95b24f55	semantic	6/6	324	0	4
glm-flash	vault_95b24f55	serena	6/6	248	0	0
glm-flash	vault_caec65a7	raw	1/6	720	6	0
glm-flash	vault_caec65a7	semantic	6/6	425	0	0
glm-flash	vault_caec65a7	serena	5/6	456	2	0

7 Limitations

One language. Contamination is held constant across arms but inflates absolute rates. One serving stack per model, small k, and the infrastructure is author-run. TODO: expand honestly.

8 Availability

Code (MIT), all episode evidence, and analysis tooling: `https://github.com/guygrigsby/agent-go`.

References

[1] Wayne Chi, Valerie Chen, Ryan Shar, Aditya Mittal, Jenny Liang, Wei-Lin Chiang, Anastasios Nikolas Angelopoulos, Ion Stoica, Graham Neubig, Ameet Talwalkar, and Chris Donahue. EDIT-Bench: Evaluating LLM abilities to perform real-world instructed code edits. `https://arxiv.org/abs/2511.04486`, 2025.

[2] Paul Gauthier. Aider polyglot coding leaderboard. `https://aider.chat/docs/leaderboards/`, 2025. Tracks percent of edits emitted in the correct format; accessed 2026-07-24.

[3] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. `https://arxiv.org/abs/2604.05407`, 2026.

[4] Beck LaBash, August Rosedale, Alex Reents, Lucas Negritto, and Colin Wiel. RES-Q: Evaluating code-editing large language model systems at the repository scale. `https://arxiv.org/abs/2406.16801`, 2024.

[5] Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. FEA-Bench: A benchmark for evaluating repository-level code generation for feature implementation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[6] Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. Live functional programming with typed holes. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019.

[7] Oraios AI. Serena: A coding agent toolkit with semantic retrieval and editing over LSP. `https://github.com/oraios/serena`, 2025. MCP server; accessed 2026-07-24.

[8] The Go Authors. gopls: Model context protocol support (experimental). `https://go.dev/gopls/features/mcp`, 2025. golang.org/x/tools/gopls, introduced in v0.20.0; accessed 2026-07-24.

[9] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.